ARCHPLAN TASK MANAGER

Technical Planning Document

NestJS · TypeScript · PostgreSQL · Domain-Driven Design

Version 1.0
February 2026
Architecture & Design Industry

Table of Contents

1. Executive Summary 3
2. System Architecture Overview 3
3. Domain-Driven Design Structure 4
4. Database Schema Design 5
5. Authentication & Authorization Module 8
6. API Endpoints Specification 10
7. Core Logic Flows 13
8. Project Setup & Configuration 16
9. Development Phases & Sprint Plan 18
10. Testing Strategy 20
11. Deployment & DevOps 21
12. Appendix: Mermaid Diagram Source 22

1. Executive Summary

This document provides a comprehensive technical plan for building ArchPlan Task Manager, a project and task management platform tailored for architecture and design firms. The application is designed as a RESTful monolithic server using NestJS with TypeScript, backed by PostgreSQL, and structured following Domain-Driven Design (DDD) principles.

The system supports recursive task hierarchies (tasks containing sub-tasks to infinite depth), real-time time tracking, multi-user task collaboration, and a role-based access control system with granular permissions. It is designed to be coupled with a React.js SPA frontend in a subsequent phase.

Key Capabilities

• Project-centric workflow with recursive task/sub-task trees

• Role-based access control (Owner, Admin, Manager, Member, Viewer)

• Per-task time tracking with start/stop/pause mechanics

• Multi-user collaboration on individual tasks

• Team-based organizational structure

• Audit trail and activity logging

2. System Architecture Overview

The application follows a layered monolithic architecture organized by DDD bounded contexts. Each bounded context encapsulates its own domain logic, application services, infrastructure layer, and presentation (controller) layer.

High-Level Architecture Diagram

Below is the layered architecture representation. Each layer has strict dependency rules: outer layers depend on inner layers, never the reverse.

Layer	Responsibility	Key Components
Presentation (Controllers)	HTTP request/response handling, input validation, route definitions	NestJS Controllers, DTOs, Swagger decorators
Application (Services)	Use case orchestration, transaction management, command/query handling	Application Services, Command Handlers, Event Handlers
Domain (Core)	Business rules, entities, value objects, domain events, aggregate roots	Entities, Value Objects, Domain Services, Repository Interfaces

Infrastructure	Database access, external services, framework integrations	TypeORM Repositories, Guards, Strategies, Mappers

Technology Stack

Technology	Version	Purpose
Node.js	20 LTS	Runtime environment
TypeScript	5.x	Type-safe development language
NestJS	10.x	Application framework with DI container
TypeORM	0.3.x	ORM with migration support
PostgreSQL	16	Primary relational database
Passport.js	0.7.x	Authentication strategies (JWT)
class-validator	0.14.x	DTO validation decorators
class-transformer	0.5.x	Object transformation
bcrypt	5.x	Password hashing
Swagger/OpenAPI	7.x	API documentation

3. Domain-Driven Design Structure

The application is decomposed into four bounded contexts, each responsible for a cohesive slice of business logic. Bounded contexts communicate through well-defined interfaces and domain events where necessary.

Bounded Contexts

Bounded Context	Aggregate Roots	Description
Identity & Access	User, Role	User registration, authentication, JWT management, role/permission assignment
Organization	Team, TeamMembership	Team creation, membership management, organizational hierarchy
Project Management	Project, Task	Project lifecycle, recursive task trees, task assignment, status management
Time Tracking	TimeEntry	Start/stop/pause time tracking per user per task, duration computation

Folder Structure

Each bounded context follows this internal layout within the src/ directory:

```
src/
  ├─ modules/
  │   ├─ identity/                # Identity & Access BC
  │   │   ├─ domain/
  │   │   │   ├─ entities/          # User.entity.ts, Role.entity.ts
  │   │   │   ├─ value-objects/       # Email.vo.ts, Password.vo.ts
  │   │   │   ├─ repositories/       # IUserRepository.ts (interface)
  │   │   │   └─ events/            # UserCreated.event.ts
  │   │   ├─ application/
  │   │   │   ├─ services/          # AuthService.ts, UserService.ts
  │   │   │   ├─ commands/           # CreateUser.command.ts
  │   │   │   └─ dtos/              # CreateUserDto.ts, LoginDto.ts
  │   │   ├─ infrastructure/
  │   │   │   ├─ persistence/        # TypeOrmUserRepository.ts
  │   │   │   ├─ guards/            # JwtAuthGuard.ts, RolesGuard.ts
  │   │   │   └─ strategies/         # JwtStrategy.ts, LocalStrategy.ts
  │   │   └─ presentation/
```

```
│   │       └─ controllers/        # AuthController.ts, UserController.ts
│   ├─ organization/              # Organization BC (same layout)
│   ├─ project-management/        # Project Management BC
│   └─ time-tracking/             # Time Tracking BC
├─ shared/
│   ├─ domain/                    # BaseEntity, AggregateRoot, DomainEvent
│   ├─ infrastructure/            # Database config, global filters, interceptors
│   └─ application/               # Pagination, sorting, shared DTOs
├─ config/                        # Environment config, validation schema
├─ database/
│   └─ migrations/                # TypeORM migrations
└─ main.ts
```

4. Database Schema Design

The database schema uses PostgreSQL with UUIDs as primary keys across all tables. Soft deletes are implemented via a deleted_at timestamp column. All entities track created_at and updated_at timestamps. The recursive task hierarchy uses a self-referencing foreign key with a materialized path for efficient tree queries.

4.1 Users Table

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT uuid_generate_v4()	Unique identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email address
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
first_name	VARCHAR(100)	NOT NULL	User first name
last_name	VARCHAR(100)	NOT NULL	User last name
avatar_url	VARCHAR(500)	NULLABLE	Profile picture URL
is_active	BOOLEAN	DEFAULT true	Account active flag
last_login_at	TIMESTAMPTZ	NULLABLE	Last login timestamp
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation time
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update time
deleted_at	TIMESTAMPTZ	NULLABLE	Soft delete marker

4.2 Roles Table

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier
name	VARCHAR(50)	UNIQUE, NOT NULL	Role name (owner, admin, manager, member, viewer)
description	VARCHAR(255)	NULLABLE	Human-readable description
is_system	BOOLEAN	DEFAULT false	Whether this is a system-defined role
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation time

4.3 Permissions Table

Column	Type	Constraints	Description

id	UUID	PK	Unique identifier
name	VARCHAR(100)	UNIQUE, NOT NULL	Permission slug (e.g. project:create)
description	VARCHAR(255)	NULLABLE	Description
resource	VARCHAR(50)	NOT NULL	Resource type (project, task, team, user)
action	VARCHAR(50)	NOT NULL	Action type (create, read, update, delete, manage)

4.4 Role-Permissions Join Table

Column	Type	Constraints	Description
role_id	UUID	PK, FK → roles.id	Role reference
permission_id	UUID	PK, FK → permissions.id	Permission reference

4.5 User-Roles Join Table

Column	Type	Constraints	Description
user_id	UUID	PK, FK → users.id	User reference
role_id	UUID	PK, FK → roles.id	Role reference
assigned_at	TIMESTAMPTZ	DEFAULT NOW()	When role was assigned
assigned_by	UUID	FK → users.id, NULLABLE	Who assigned the role

4.6 Teams Table

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier
name	VARCHAR(100)	NOT NULL	Team name
description	TEXT	NULLABLE	Team description
created_by	UUID	FK → users.id	Team creator
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation time
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update time
deleted_at	TIMESTAMPTZ	NULLABLE	Soft delete marker

4.7 Team Memberships Table

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier
team_id	UUID	FK → teams.id, NOT NULL	Team reference
user_id	UUID	FK → users.id, NOT NULL	User reference
team_role	ENUM	NOT NULL (lead, member)	Role within the team
joined_at	TIMESTAMPTZ	DEFAULT NOW()	When user joined
UNIQUE		(team_id, user_id)	One membership per user per team

4.8 Projects Table

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier
name	VARCHAR(200)	NOT NULL	Project name
description	TEXT	NULLABLE	Project description
status	ENUM	NOT NULL, DEFAULT 'planning'	planning, active, on_hold, completed, archived
start_date	DATE	NULLABLE	Project start date
due_date	DATE	NULLABLE	Project deadline
team_id	UUID	FK → teams.id, NOT NULL	Owning team
created_by	UUID	FK → users.id	Project creator
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation time
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update time

deleted_at	TIMESTAMPTZ	NULLABLE	Soft delete marker

4.9 Tasks Table (Recursive Hierarchy)

The tasks table supports infinite nesting via a self-referencing parent_id and a materialized_path column for efficient ancestor/descendant queries. The materialized_path stores the full path of ancestor IDs separated by dots (e.g., "uuid1.uuid2.uuid3").

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier
title	VARCHAR(300)	NOT NULL	Task title
description	TEXT	NULLABLE	Task description / brief
status	ENUM	NOT NULL, DEFAULT 'todo'	todo, in_progress, in_review, done, cancelled
priority	ENUM	DEFAULT 'medium'	low, medium, high, urgent
start_date	DATE	NULLABLE	Planned start date
due_date	DATE	NULLABLE	Planned due date
parent_id	UUID	FK → tasks.id, NULLABLE	Parent task (NULL = top-level)
materialized_path	VARCHAR(3000)	NOT NULL, DEFAULT ''	Dot-separated ancestor path
depth	INTEGER	NOT NULL, DEFAULT 0	Nesting depth (0 = top-level)
position	INTEGER	NOT NULL, DEFAULT 0	Sort order among siblings
project_id	UUID	FK → projects.id, NOT NULL	Owning project
created_by	UUID	FK → users.id	Task creator
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation time
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update time
deleted_at	TIMESTAMPTZ	NULLABLE	Soft delete marker

Index recommendations: CREATE INDEX idx_tasks_parent ON tasks(parent_id); CREATE INDEX idx_tasks_path ON tasks USING GIN(materialized_path gin_trgm_ops); CREATE INDEX idx_tasks_project ON tasks(project_id); CREATE INDEX idx_tasks_status ON tasks(project_id, status);

4.10 Task Assignees Table

Supports multiple users assigned to a single task simultaneously.

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier

task_id	UUID	FK → tasks.id, NOT NULL	Task reference
user_id	UUID	FK → users.id, NOT NULL	Assigned user
assigned_at	TIMESTAMPTZ	DEFAULT NOW()	When assigned
assigned_by	UUID	FK → users.id	Who assigned
UNIQUE		(task_id, user_id)	One assignment per user per task

4.11 Time Entries Table

Tracks individual time tracking sessions. A NULL end_time indicates the timer is currently running.

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier
task_id	UUID	FK → tasks.id, NOT NULL	Task being tracked
user_id	UUID	FK → users.id, NOT NULL	User tracking time
start_time	TIMESTAMPTZ	NOT NULL	When timer started
end_time	TIMESTAMPTZ	NULLABLE	When timer stopped (NULL = running)
duration_seconds	INTEGER	NULLABLE	Computed duration in seconds
description	VARCHAR(500)	NULLABLE	Optional work description
is_manual	BOOLEAN	DEFAULT false	Whether manually entered
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation time
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update time

Constraint: CHECK (end_time IS NULL OR end_time > start_time). Partial unique index: CREATE UNIQUE INDEX idx_one_active_timer ON time_entries(user_id) WHERE end_time IS NULL; — ensures each user can only have one running timer at a time.

4.12 Activity Log Table

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier
actor_id	UUID	FK → users.id	User who performed the action
action	VARCHAR(50)	NOT NULL	Action type (created, updated, deleted, assigned, etc.)
entity_type	VARCHAR(50)	NOT NULL	Target entity type (project, task, team)
entity_id	UUID	NOT NULL	Target entity ID
metadata	JSONB	NULLABLE	Additional context (old/new values, etc.)
created_at	TIMESTAMPTZ	DEFAULT NOW()	When the action occurred

4.13 Entity Relationship Diagram

The following describes the entity relationships. A visual ER diagram in Mermaid format is provided in the appendix and as a separate .mermaid file.

- users (1) ── (M) user_roles (M) ── (1) roles
- roles (1) ── (M) role_permissions (M) ── (1) permissions
- users (1) ── (M) team_memberships (M) ── (1) teams
- teams (1) ── (M) projects
- projects (1) ── (M) tasks
- tasks (1) ── (M) tasks [self-referencing, parent_id]
- tasks (1) ── (M) task_assignees (M) ── (1) users
- tasks (1) ── (M) time_entries (M) ── (1) users
- All entities ── (M) activity_log

5. Authentication & Authorization Module

5.1 Authentication Flow

The system uses JWT-based authentication with access and refresh tokens. Access tokens have a short lifespan (15 minutes) and refresh tokens are longer-lived (7 days). The refresh token is stored as an HTTP-only cookie for security.

Registration Flow: User submits email + password → Validate uniqueness → Hash password with bcrypt (12 rounds) → Create user record → Assign default 'member' role → Return JWT pair.

Login Flow: User submits credentials → Validate email exists → Compare bcrypt hash → Generate access token (15min) + refresh token (7d) → Set refresh token as HTTP-only cookie → Return access token in body.

Token Refresh: Client sends refresh token cookie → Validate token → Check user is active → Issue new access + refresh tokens → Rotate refresh token (invalidate old).

5.2 Role Hierarchy

Roles follow a hierarchical permission model. Higher roles inherit all permissions of lower roles.

Role	Level	Scope	Key Permissions
Owner	5	Global	All permissions, system configuration, role management, user management
Admin	4	Global	Manage all projects/teams, manage users (except owners), view all data
Manager	3	Team-scoped	Create/manage projects for their teams, assign tasks, manage team members
Member	2	Project-scoped	Create/edit tasks, track time, view assigned projects, comment
Viewer	1	Project-scoped	Read-only access to assigned projects and tasks

5.3 Permission Matrix

Permission	Owner	Admin	Manager	Member	Viewer
user:create	✓	✓	✗	✗	✗
user:update	✓	✓	✗ (self only)	✗ (self only)	✗ (self only)
user:delete	✓	✓	✗	✗	✗

role:assign	✓	✓ (below admin)	✗	✗	✗
team:create	✓	✓	✗	✗	✗
team:manage	✓	✓	✓ (own teams)	✗	✗
project:create	✓	✓	✓	✗	✗
project:update	✓	✓	✓ (own teams)	✗	✗
project:delete	✓	✓	✓ (own teams)	✗	✗
task:create	✓	✓	✓	✓	✗
task:update	✓	✓	✓	✓ (assigned)	✗
task:delete	✓	✓	✓	✗	✗
task:assign	✓	✓	✓	✗	✗
time:track	✓	✓	✓	✓	✗
time:manage	✓	✓	✓	✗ (own only)	✗
report:view	✓	✓	✓ (own teams)	✗	✗

5.4 Authorization Implementation

Authorization is enforced at two levels:

1. Guard-Level (Coarse): A custom @Roles() decorator combined with a RolesGuard checks if the authenticated user has the minimum required role level for the endpoint. Applied via NestJS guards on controller methods.

2. Policy-Level (Fine): A PermissionsGuard checks specific permissions. For resource-scoped permissions (e.g., a manager can only update their own team's projects), the guard loads the resource and verifies the user's relationship to it. This uses a CASL-inspired ability factory pattern.

Implementation pattern: @UseGuards(JwtAuthGuard, RolesGuard) @Roles('manager') @Permissions('project:update') updateProject(). The JwtAuthGuard verifies the token, RolesGuard checks role level, and PermissionsGuard checks the specific permission plus resource ownership.

6. API Endpoints Specification

All endpoints are prefixed with /api/v1. Responses follow a consistent envelope format: { data, meta, errors }. Pagination uses cursor-based pagination for lists.

6.1 Authentication Endpoints

Method	Path	Auth	Description
POST	/auth/register	Public	Register new user account
POST	/auth/login	Public	Authenticate and receive JWT tokens
POST	/auth/refresh	Cookie	Refresh access token using refresh token
POST	/auth/logout	JWT	Invalidate refresh token
POST	/auth/forgot-password	Public	Request password reset email
POST	/auth/reset-password	Token	Reset password with token

6.2 User Endpoints

Method	Path	Auth	Description
GET	/users	Admin+	List all users (paginated, filterable)
GET	/users/:id	JWT	Get user profile (self or admin)
PATCH	/users/:id	JWT	Update user profile
DELETE	/users/:id	Admin+	Soft delete user account
GET	/users/me	JWT	Get current user profile
PATCH	/users/me/password	JWT	Change own password
GET	/users/:id/roles	Admin+	Get user roles
POST	/users/:id/roles	Admin+	Assign role to user
DELETE	/users/:id/roles/:roleId	Admin+	Remove role from user

6.3 Team Endpoints

Method	Path	Auth	Description
POST	/teams	Admin+	Create a new team
GET	/teams	JWT	List teams (filtered by membership for non-admins)
GET	/teams/:id	JWT	Get team details
PATCH	/teams/:id	Manager+	Update team info
DELETE	/teams/:id	Admin+	Soft delete team

GET	/teams/:id/members	JWT	List team members
POST	/teams/:id/members	Manager+	Add member to team
PATCH	/teams/:id/members/:userId	Manager+	Update member role in team
DELETE	/teams/:id/members/:userId	Manager+	Remove member from team

6.4 Project Endpoints

Method	Path	Auth	Description
POST	/projects	Manager+	Create project for a team
GET	/projects	JWT	List projects (scoped by team access)
GET	/projects/:id	JWT	Get project details with stats
PATCH	/projects/:id	Manager+	Update project
DELETE	/projects/:id	Manager+	Soft delete project
GET	/projects/:id/tasks	JWT	Get top-level tasks for project
GET	/projects/:id/tasks/tree	JWT	Get full task tree (recursive)

6.5 Task Endpoints

Method	Path	Auth	Description
POST	/tasks	Member+	Create task (top-level or sub-task)
GET	/tasks/:id	JWT	Get task with children summary
PATCH	/tasks/:id	Member+	Update task details/status
DELETE	/tasks/:id	Manager+	Soft delete task (cascades to children)
GET	/tasks/:id/children	JWT	Get direct children of a task
GET	/tasks/:id/subtree	JWT	Get full subtree below a task
PATCH	/tasks/:id/move	Member+	Move task to new parent or reorder
GET	/tasks/:id/assignees	JWT	List task assignees
POST	/tasks/:id/assignees	Manager+	Assign user(s) to task
DELETE	/tasks/:id/assignees/:userId	Manager+	Unassign user from task

6.6 Time Tracking Endpoints

Method	Path	Auth	Description
POST	/tasks/:taskId/time/start	Member+	Start time tracker on a task
POST	/tasks/:taskId/time/stop	Member+	Stop running timer
GET	/tasks/:taskId/time	JWT	Get time entries for a task
GET	/users/me/time	JWT	Get current user time entries
GET	/users/me/time/active	JWT	Get currently running timer
POST	/time-entries	Member+	Create manual time entry
PATCH	/time-entries/:id	JWT	Update time entry (own only)
DELETE	/time-entries/:id	JWT	Delete time entry (own or manager+)
GET	/projects/:id/time-report	Manager+	Get aggregated time report for project

6.7 Activity Log Endpoints

Method	Path	Auth	Description
GET	/projects/:id/activity	JWT	Get activity log for a project
GET	/tasks/:id/activity	JWT	Get activity log for a task
GET	/activity	Admin+	Get global activity feed (paginated)

7. Core Logic Flows

This section details the key business logic flows. Sequence diagrams in Mermaid format are provided in the appendix.

7.1 Task Creation Flow

Creating a task (either top-level or sub-task) follows these steps:

Step 1 — Validate Input: Controller receives CreateTaskDto with title, description, start_date, due_date, project_id, and optional parent_id. class-validator validates all fields.

Step 2 — Authorization Check: PermissionsGuard verifies the user has task:create permission and belongs to the project's team.

Step 3 — Parent Resolution: If parent_id is provided, load the parent task. Verify it belongs to the same project. Compute new materialized_path = parent.materialized_path + '.' + parent.id. Set depth = parent.depth + 1.

Step 4 — Position Calculation: Query the max position among siblings (same parent_id and project_id). Set position = max + 1.

Step 5 — Entity Creation: Create Task domain entity with all computed fields. Persist via repository.

Step 6 — Domain Event: Emit TaskCreated domain event. Activity log subscriber records the action.

Step 7 — Response: Return the created task with its computed path and depth.

7.2 Recursive Task Tree Retrieval

Fetching the full task tree for a project uses two strategies depending on dataset size:

Strategy A — Materialized Path Query: SELECT * FROM tasks WHERE project_id = :projectId AND materialized_path LIKE :rootPath% ORDER BY materialized_path, position. This retrieves all descendants in a single query. The application layer then assembles the tree structure in memory by iterating sorted results and building parent-child relationships using a HashMap.

Strategy B — Recursive CTE (for subtree queries): WITH RECURSIVE task_tree AS (SELECT * FROM tasks WHERE id = :taskId UNION ALL SELECT t.* FROM tasks t JOIN task_tree tt ON t.parent_id = tt.id) SELECT * FROM task_tree ORDER BY depth, position. Used when fetching a subtree from a specific node.

7.3 Time Tracking Flow

Start Timer: User calls POST /tasks/:taskId/time/start. System checks: (1) User is assigned to the task or has manager+ role. (2) User has no other running timer (enforced by partial unique index). If a timer is already running on another task, return a 409 Conflict with the active timer details so the frontend can offer to stop it. Create TimeEntry with start_time = NOW(), end_time = NULL.

Stop Timer: User calls POST /tasks/:taskId/time/stop. System finds the active TimeEntry for this user and task (end_time IS NULL). Sets end_time = NOW(). Computes duration_seconds = EXTRACT(EPOCH FROM end_time - start_time). Updates the record. Emits TimeEntryStopped event.

Edge Cases: (1) Stopping a timer on a different task than where one is running returns 404. (2) Starting a timer auto-suggests stopping the current one. (3) Manual entries bypass the running timer constraint. (4) Duration for running timers is computed on-read as NOW() - start_time.

7.4 Task Status Transition Flow

Task status follows a state machine with defined valid transitions:

- todo → in_progress, cancelled
- in_progress → in_review, todo, cancelled
- in_review → done, in_progress
- done → in_progress (reopen)
- cancelled → todo (restore)

When a parent task's children all reach 'done' status, the system suggests (but does not auto-transition) the parent to 'done'. This is communicated via the response metadata. When a parent task moves to 'cancelled', all non-completed children are also moved to 'cancelled' with a cascading domain event.

7.5 Task Assignment Flow

Assign: Manager calls POST /tasks/:id/assignees with { userIds: [...] }. System verifies each user is a member of the project's team. Creates TaskAssignee records. Emits TaskAssigned event for each user. Activity log records the assignments.

Multi-User Collaboration: Multiple users can be assigned to the same task. Each tracks their own time independently. The task view aggregates total time from all assignees. Assignees can see each other's time entries on the task for coordination.

7.6 Authentication Flow Diagram (Textual)

Login Sequence:

Client → POST /auth/login {email, password}
AuthController → AuthService.login()
AuthService → UserRepository.findByEmail()
AuthService → bcrypt.compare(password, hash)
AuthService → JwtService.sign(accessPayload, {expiresIn: '15m'})
AuthService → JwtService.sign(refreshPayload, {expiresIn: '7d'})
AuthService → Store refreshToken hash in DB
AuthController → Set refresh cookie (HttpOnly, Secure, SameSite)
AuthController → Return { accessToken, user } to Client

Protected Request Sequence:

Client → GET /projects (Authorization: Bearer <accessToken>)
JwtAuthGuard → JwtStrategy.validate(payload)
JwtStrategy → UserRepository.findById(payload.sub)
RolesGuard → Check user.roles against @Roles() metadata
PermissionsGuard → Load resource, check AbilityFactory rules
Controller → Execute handler, return response

8. Project Setup & Configuration

8.1 Initial Setup Commands

Execute the following to scaffold the project:

```
# Install NestJS CLI globally
npm install -g @nestjs/cli

# Create the project
nest new archplan-api --strict --package-manager npm

# Install core dependencies
npm install @nestjs/typeorm typeorm pg
npm install @nestjs/passport passport passport-jwt passport-local
npm install @nestjs/jwt jsonwebtoken
npm install @nestjs/swagger swagger-ui-express
npm install class-validator class-transformer
npm install bcrypt uuid
npm install @nestjs/config joi
npm install @nestjs/event-emitter

# Install dev dependencies
npm install -D @types/passport-jwt @types/passport-local @types/bcrypt
npm install -D @types/uuid
```

8.2 Environment Configuration

Create a .env file with the following variables:

Variable	Example Value	Description
NODE_ENV	development	Environment (development, staging, production)
PORT	3000	Server port
DB_HOST	localhost	PostgreSQL host
DB_PORT	5432	PostgreSQL port
DB_USERNAME	archplan_user	Database user

DB_PASSWORD	secure_password	Database password
DB_DATABASE	archplan_db	Database name
JWT_ACCESS_SECRET	<random-64-char>	Access token signing secret
JWT_REFRESH_SECRET	<random-64-char>	Refresh token signing secret
JWT_ACCESS_EXPIRY	15m	Access token lifetime
JWT_REFRESH_EXPIRY	7d	Refresh token lifetime
BCRYPT_ROUNDS	12	Bcrypt hashing cost factor
CORS_ORIGIN	http://localhost:3001	Allowed CORS origin (React SPA)

8.3 TypeORM Configuration

Configure TypeORM in AppModule with the following settings. Use migrations (not synchronize) for production safety:

```
TypeOrmModule.forRootAsync({
  imports: [ConfigModule],
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    type: 'postgres',
    host: config.get('DB_HOST'),
    port: config.get('DB_PORT'),
    username: config.get('DB_USERNAME'),
    password: config.get('DB_PASSWORD'),
    database: config.get('DB_DATABASE'),
    entities: [__dirname + '/**/*.entity{.ts,.js}'],
    migrations: [__dirname + '/database/migrations/*{.ts,.js}'],
    synchronize: false,   // NEVER true in production
    logging: config.get('NODE_ENV') === 'development',
    ssl: config.get('NODE_ENV') === 'production' ? { rejectUnauthorized: false } : false,
  }),
})
```

8.4 Database Seeding Strategy

Create a seed script (src/database/seeds/) that runs on first deployment to populate: (1) System roles (owner, admin, manager, member, viewer) with is_system = true. (2) All permissions from the permission matrix. (3) Role-permission mappings. (4) A default owner account. The seed should be idempotent (safe to run multiple times).

9. Development Phases & Sprint Plan

The project is divided into 5 phases across approximately 10 two-week sprints. Each phase builds upon the previous, allowing for incremental testing and deployment.

Phase 1: Foundation (Sprints 1–2)

Sprint	Task	Deliverable	Est. Hours
S1	Project scaffolding, config, DB setup	Running NestJS app with DB connection	16
S1	Shared domain base classes (BaseEntity, AggregateRoot)	Shared module with base infrastructure	8
S1	User entity, repository, basic CRUD	User management endpoints	16
S1	Role & Permission entities, seeding	Seeded RBAC data	12
S2	JWT auth (register, login, refresh, logout)	Full auth flow with tokens	20
S2	Auth guards (JWT, Roles, Permissions)	Working authorization middleware	16
S2	Password reset flow	Email-triggered password reset	8
S2	Unit tests for auth module	90%+ coverage on auth	12

Phase 2: Organization (Sprints 3–4)

Sprint	Task	Deliverable	Est. Hours
S3	Team entity, CRUD endpoints	Team management API	16
S3	Team membership management	Add/remove/update members	12
S3	Team-scoped authorization rules	Managers can only manage own teams	12
S4	Project entity, CRUD endpoints	Project management API	16
S4	Project-team relationship and scoping	Projects bound to teams	8
S4	Project status management	Status transitions with validation	8
S4	Integration tests for org module	E2E tests for teams + projects	16

Phase 3: Task Management (Sprints 5–7)

Sprint	Task	Deliverable	Est. Hours

S5	Task entity with recursive hierarchy	Task CRUD with parent/child	20
S5	Materialized path computation + indexes	Efficient tree queries	12
S5	Task tree retrieval (flat + nested)	GET /projects/:id/tasks/tree	12
S6	Task status state machine	Validated status transitions	10
S6	Task assignment (multi-user)	Assign/unassign endpoints	12
S6	Task move/reorder logic	PATCH /tasks/:id/move	16
S7	Cascading operations (delete, cancel)	Parent-child cascade logic	12
S7	Task filtering, sorting, pagination	Query params on list endpoints	12
S7	Integration tests for task module	Full task lifecycle E2E tests	16

Phase 4: Time Tracking (Sprint 8)

Sprint	Task	Deliverable	Est. Hours
S8	TimeEntry entity and repository	Time tracking data layer	8
S8	Start/stop timer endpoints	Real-time time tracking	12
S8	Active timer constraint (one per user)	Partial unique index + validation	6
S8	Manual time entry CRUD	Manual entry endpoints	8
S8	Time aggregation queries	Per-task, per-project, per-user reports	12
S8	Time tracking tests	Full timer lifecycle tests	8

Phase 5: Polish & Production Prep (Sprints 9–10)

Sprint	Task	Deliverable	Est. Hours
S9	Activity log system (events + persistence)	Audit trail for all actions	16
S9	Global error handling + response formatting	Consistent API responses	8
S9	Swagger/OpenAPI documentation	Interactive API docs	8
S9	Rate limiting + security headers	Production security hardening	8
S10	Performance optimization + query tuning	Optimized DB queries, indexes	16
S10	Docker containerization	Dockerfile + docker-compose	8
S10	CI/CD pipeline setup	GitHub Actions or equivalent	8
S10	Final integration + load testing	Production-ready verification	16

Estimated Total

Phase	Sprints	Estimated Hours
Phase 1: Foundation	1–2	108
Phase 2: Organization	3–4	88
Phase 3: Task Management	5–7	122
Phase 4: Time Tracking	8	54
Phase 5: Polish & Production	9–10	88
TOTAL	1–10	460

10. Testing Strategy

10.1 Testing Layers

Layer	Tool	Coverage Target	What to Test
Unit Tests	Jest	80%+ domain/app layers	Domain entities, value objects, services, guards, pipes
Integration Tests	Jest + Supertest	All API endpoints	Controller → DB round trips, auth flows, cascading logic
E2E Tests	Jest + Supertest	Critical user journeys	Full registration → project → task → time tracking flow
DB Tests	Jest + pg-mem or testcontainers	Migrations, seeds	Migration up/down, seed idempotency, constraint checks

10.2 Test Database Strategy

Use a separate test database that is created fresh for each test suite run. Each test file wraps operations in transactions that are rolled back after each test to ensure isolation. For integration tests, use testcontainers/postgresql to spin up a real PostgreSQL instance in Docker.

10.3 Key Test Scenarios

Auth: Registration with duplicate email fails. Login returns valid JWT. Expired tokens are rejected. Refresh token rotation works. Role-based access is enforced.

Tasks: Creating a sub-task correctly computes materialized_path and depth. Deleting a parent cascades to children. Moving a task recomputes paths for entire subtree. Invalid status transitions are rejected. Task tree retrieval returns correct nested structure.

Time Tracking: Starting a timer while another is running returns 409. Stopping a timer computes correct duration. Only one active timer per user is enforced at the database level. Manual entries do not conflict with running timers.

Authorization: Members cannot create projects. Managers can only manage their own teams' resources. Viewers have read-only access. Role inheritance works correctly.

11. Deployment & DevOps

11.1 Docker Setup

The application uses a multi-stage Docker build for optimized image size. The Dockerfile builds TypeScript, then copies only compiled JS and production node_modules to the runtime image. A docker-compose.yml orchestrates the API server, PostgreSQL database, and optionally pgAdmin for database management.

docker-compose.yml services:

- api: Node.js app (port 3000), depends on db, healthcheck on /api/health
- db: PostgreSQL 16 (port 5432), persistent volume for data
- pgadmin: pgAdmin 4 (port 5050), for development only

11.2 CI/CD Pipeline

Recommended GitHub Actions workflow with the following stages:

Stage	Trigger	Actions
Lint & Type Check	Every push	ESLint, TypeScript compiler (--noEmit)
Unit Tests	Every push	Jest unit test suite with coverage report
Integration Tests	Pull request	Start test DB, run migration, run integration tests
Build	Merge to main	Docker build, tag with git SHA
Deploy Staging	Merge to main	Push image to registry, deploy to staging
Deploy Production	Manual trigger / tag	Deploy to production with migration

11.3 Database Migration Strategy

All schema changes go through TypeORM migrations, never synchronize. Migrations run automatically on deployment before the application starts. Each migration is tested with both up and down operations. Naming convention: <timestamp>-<descriptive-name>.ts (e.g., 1709123456-create-users-table.ts). Destructive migrations (column drops, table drops) require a two-phase approach: first deprecate in one release, then remove in the next.

12. Appendix: Diagram Sources

The following Mermaid diagram source code files are provided alongside this document for rendering in any Mermaid-compatible tool (GitHub, VS Code extension, mermaid.live, etc.)

12.1 Entity Relationship Diagram

File: er-diagram.mermaid

This diagram shows the full database schema with all tables, columns, types, and relationships including the self-referencing task hierarchy, many-to-many join tables, and foreign key relationships.

12.2 Authentication Sequence Diagram

File: auth-flow.mermaid

This diagram illustrates the login and protected request flows including JWT generation, refresh token rotation, guard chain execution, and error paths.

12.3 Task Creation Sequence Diagram

File: task-creation-flow.mermaid

This diagram details the step-by-step task creation process including parent resolution, materialized path computation, position calculation, and domain event emission.

12.4 Time Tracking Sequence Diagram

File: time-tracking-flow.mermaid

This diagram covers the start timer, stop timer, and conflict resolution flows including the active timer check, duration computation, and edge case handling.

12.5 Task Status State Machine

File: task-status-states.mermaid

This state diagram shows all valid task status transitions, including reopen and restore paths, with guard conditions noted on each transition edge.